

Assignment 2 Report: Leader Election and Following multi-agent systems

Group 7

Name	Email
Bernardo Tavares Monteiro Fernandes Portugal	au804038@uni.au.dk
Mohamed Hashem Abdou	au656408@uni.au.dk
Rafael Ferreira da Costa Silva Valqueresma	au804039@uni.au.dk
Yu Jin	au785458@uni.au.dk

Task Overview

Basic Task:
Assume a MAS with 5 agents. Assume a leader agent (pre-assigned, and identifiable by the rest) and the rest follower agents. Implement a nature-inspired mechanism (either ant or bird inspired) that enables the agents to follow their leader, as well as keep some distance between each other (in a straight line or some other formation). No direct communication is allowed between agents. Define the quality metrics and evaluate your method

Advanced Task:
Implement a leader election mechanism (e.g., bully). Have agents communicate to decide which position in the fleet to take, use one of the mentioned coordination mechanisms. Scale the number of agents to, 10, 20 etc. Define the quality metrics and evaluate your method

Solution Summary

Basic Task: ACO Leader-Following

Our solution for the basic task is based on Ant Colony Optimization (ACO), an ant-inspired algorithm that uses indirect communication via environmental markers (pheromones).

- **Implemented in:**
 - Basic Implementation `assignment2/ACO.py`
 - test with: `assignment2/test1.py`
- **Mechanism (Indirect Communication):**
 - A global 2D grid simulates a **pheromone field**.
 - The **leader agent** moves along a predefined waypoint path, depositing a high concentration of pheromones (`PH_DEP_L`) to create a strong scent trail.
 - **Follower agents** are attracted to this trail and deposit a smaller amount of pheromones (`PH_DEP_F`) to reinforce the path.
 - Pheromones gradually **evaporate** (`PH_EVAP`) and **diffuse** (`PH_DIFF`), ensuring that the trail stays fresh and followers react to the leader's current path.
- **Follower Policy:** Followers choose their direction by balancing two main factors for multiple candidate headings:
 1. **Pheromone Strength (τ):** The intensity of the scent trail in a given direction.
 2. **Heuristic Value (η):** The desirability of a heading based on immediate sensor data. Our heuristic combines:
 - **Obstacle Clearance:** Distance to obstacles detected by the lidar sensor.
 - **Attraction to Leader:** Proximity to the leader's current position.

A direction is probabilistically selected using the standard ACO formula: $score = \tau^{\alpha} * \eta^{\beta}$, which effectively merges trail-following with goal-seeking behavior.
- **Collision Avoidance:** All agents run a high-priority safety behavior that uses lidar data to calculate **Time-To-Collision (TTC)**. If an obstacle or another agent is too close, the agent will automatically slow down, stop, or execute an escape maneuver, ensuring agent and group safety.

Quality Metrics & Results

To evaluate the performance of our ACO-based flocking system, we measure the following metrics, which are reported at the end of each simulation:

- **mean_dist_to_leader** : The average distance of all follower agents from the leader. A lower value indicates a tighter, more cohesive formation.
- **mean_inter_member_dist** : The average distance between any two agents in the swarm. This helps quantify the formation's spacing.
- **follower_onpath_ratio** : The percentage of time that followers spend within a defined band (`PATH_BAND = 0.5m`) around the leader's historical path. A higher value signifies better path fidelity.

- `mean_pheromone_conc` : The average pheromone intensity experienced by followers. This can indicate how well the followers are "on the trail."

Experiment 1: Higher Beta than Alpha (This means agents prioritize moving towards the leader's *current* location (leader attraction) over following the historical path.)

- Goal:** Evaluate the system behavior using the default parameters from `ACO.py` .

System Parameters:

Parameter	Value	Description
ALPHA	2.0	Weight of the pheromone trail (τ)
BETA	1.0	Weight of the heuristic (η)
ETA_W_LEADER	0.8	Weight for leader attraction in the heuristic
PH_EVAP	0.99	Pheromone retention rate per step (1% evaporation)
<code>vn</code> (Follower Speed)	0.75	Nominal forward speed scale for follower agents

Observed Results (2000 steps):

Metric	Value
<code>mean_dist_to_leader</code>	4.002
<code>mean_inter_member_dist</code>	1.742
<code>follower_onpath_ratio</code>	0.0
<code>mean_pheromone_conc</code>	20.188

Commentary on Results:

The results from our baseline test are a direct reflection of the chosen system parameters:

- Loose Formation (`mean_dist_to_leader` : 4.002):** The average distance to the leader is very high. This is primarily because the followers' nominal speed (`vn` = 0.75) is significantly lower than the leader's speed (1.0), causing them to fall behind consistently.
- Heuristic Dominance:** The parameters `ALPHA` = 1.0 and `BETA` = 2.5 place a much stronger emphasis on the heuristic component (η) than on the pheromone trail (τ). This means agents prioritize moving towards the leader's *current* location (leader attraction) over following the historical path.
- Complete Failure of Path Following (`follower_onpath_ratio` : 0.0):** This metric is the most telling consequence of the parameter settings. Because the heuristic (`BETA`) is dominant and followers are slow, they are constantly trying to "cut corners" towards the distant leader instead of following the trail. They never manage to get on the path, so the pheromone component of their decision-making becomes completely irrelevant. The system is not behaving like an ant colony but rather like a simple "move-towards-leader" system.

Experiment 2: Balanced Trail-Following (High Alpha, Moderate Beta)

- Goal:** Evaluate system performance with parameters balanced to favor trail-following (`ALPHA`) while providing a moderate heuristic (`BETA`) to help lost agents recover.

System Parameters:

Parameter	Value	Description
ALPHA	2.5	High weight for the pheromone trail (τ)
BETA	1.5	Moderate weight for the heuristic (η)
ETA_W_LEADER	1.1	Moderate weight for leader attraction
PH_EVAP	0.99	Pheromone retention rate per step (1% evaporation)
<code>vn</code> (Follower Speed)	0.75	Nominal forward speed scale for follower agents

Observed Results (2000 steps):

Metric	Value
mean_dist_to_leader	4.700
mean_inter_member_dist	2.415
follower_onpath_ratio	0.0
mean_pheromone_conc	22.609

Commentary on Results:

This experiment aimed to fix the issues from the previous tests by creating a more balanced agent policy. The hypothesis was that a strong pheromone weight (ALPHA) would ensure trail-following, while a moderate heuristic (BETA) would act as a "leash" to guide lost agents back to the trail.

Analysis of system0: The continued failure, even with theoretically better parameters, points to a fundamental initialization or sensitivity problem in the system.

- Initial State is Critical:** At the beginning of the simulation, the followers start "off the trail." The moderate heuristic was intended to guide them onto it. The fact that they never succeeded suggests that by the time they get close, the original pheromone trail has already evaporated too much to be detected strongly.
- System:** This demonstrates the fragility of a purely pheromone-based system. It relies on agents staying in continuous contact with the trail. If contact is ever broken (due to speed differences, obstacles, or poor initial position), the agent may not have a robust enough mechanism to recover and find the trail again.

Overall Conclusion from Experiments: Across all tested parameter configurations, the agents have consistently failed to follow the leader's path as intended by the ACO model. This is not a failure of the algorithm's theory but an illustration of its practical challenges. The system is highly sensitive to initial conditions and the interplay between agent speed, evaporation rates, and the strength of the recovery heuristic. For this system to work, the agents must be able to acquire the trail very early in the simulation and be fast enough to never lose it.

Advanced Task: Bully Election & Auction-Based Formation

For the advanced task, we implemented a system with direct communication to solve leadership and coordination explicitly. This system uses two well-known distributed algorithms.

- Implemented in:**
 - Basic Implementation [assignment2/bully_FPSB.py](#)
 - test with: [assignment2/test_bully.py](#)
- Leader Election (Bully Algorithm):**
 - Goal:** To ensure the agent with the highest ID always becomes the leader, and to elect a new leader if the current one fails.
 - Mechanism:** If an agent detects the leader is missing (no HEARTBEAT messages), it initiates an election by messaging all agents with higher IDs. If it receives no reply, it declares itself the new leader via a VICTORY broadcast. If a higher-ID agent responds, that agent takes over the election process. This guarantees a deterministic leader without a central authority.
- Position Coordination (First-Price Sealed-Bid Auction):**
 - Goal:** To have follower agents decide which position in a V-formation to take.
 - Mechanism:**
 - The leader broadcasts an AUC_REQ (Auction Request) for all available slots in the formation.
 - Each follower calculates a "bid" for each slot, where the bid price is higher for closer slots ($price = 1 / distance$).
 - The leader collects all bids and assigns each slot to the highest bidder, announcing the results via an ASSIGN message.
 - This market-based approach allows agents to dynamically and efficiently sort themselves into an optimal formation based on proximity.

Advanced Task: Quality Metrics & Results

To evaluate the advanced system, we introduced new metrics to measure the performance of the election and coordination protocols.

- avg_election_time**: The average time from the start of an election until a VICTORY message is broadcast. A lower value indicates a more efficient election process.
- mean_pos_error**: The average distance between a follower and its assigned target coordinate in the V-formation. This is the most important metric for evaluating formation accuracy.
- mean_dist_to_leader**: Average distance from followers to the leader.
- mean_inter_member_dist**: Average distance between agents.

Experiment 1: Scaling to 10 Agents

- Goal:** Evaluate the system's performance with a medium-sized fleet of 10 agents, testing the efficiency of the election and the precision of the auction-based formation. In this experiment, the leader's speed is shared directly with the followers ($SPEED = abs(v)$).

Observed Results (10 Agents, Matched Speed):

Metric	Value

Metric	Value
avg_election_time	1.743
mean_pos_error	3.385
mean_dist_to_leader	3.772
mean_inter_member_dist	2.162

Commentary on Results:

The results from the 10-agent simulation show that while the high-level coordination protocols are working, the low-level control and formation-holding are struggling. By setting the leader and follower speeds to be the same, we observed a marked improvement in the system's ability to hold a formation.

- Significant Formation Accuracy (mean_pos_error : 3.385):** This directly confirms our hypothesis: when followers are physically capable of keeping up with the leader, they are much more successful at reaching and maintaining their assigned slots.
- Tighter Formation (mean_dist_to_leader : 3.772):** As a direct consequence of the improved position error, the average distance to the leader also decreased, indicating a more cohesive group.
- Remaining Positional Error:** While much improved, a mean_pos_error of 3.385 is still substantial. This remaining error is likely not due to speed but to other factors:
 - Controller Dynamics:** The _ctrl_to_point function is a simple proportional controller. It may not be aggressive enough to completely eliminate the error, especially when the target slot is constantly moving along a curve.
 - Collision Avoidance:** The safety system (_collision_avoidance_vel) will override the formation-following commands to prevent agents from getting too close. This necessary safety behavior forces agents to deviate from their ideal target position, contributing to the overall error.

Conclusion from Matched Speed Test: This experiment successfully demonstrates that the high-level coordination (election and auction) works correctly, but its effectiveness is fundamentally limited by the physical capabilities of the agents (their speed). When speeds are matched, the formation is significantly more stable. The remaining error can be attributed to the inherent challenges of dynamic control and safety overrides in a dense multi-agent system.

Experiment 2: Scaling to 20 Agents with Leader Escape

- Goal:** To test the system's scalability with 20 agents and to verify the leader's ability to "escape" the formation by moving faster than the followers ($SPEED = abs(v) * 0.75$), because we tested it with the same velocity and the leader would be stuck in the middle of the followers if it was not faster.

Observed Results (20 Agents, Mismatched Speed):

Metric	Value
avg_election_time	2.047
mean_pos_error	5.104
mean_dist_to_leader	5.497
mean_inter_member_dist	2.465

Commentary on Results:

This experiment highlights the costs associated with scaling up the number of agents in the fleet.

- Increased Election Time (avg_election_time : 2.047s):** The time to elect a leader more than doubled compared to the 10-agent scenario. This is an expected characteristic of the Bully algorithm. With more agents, an election initiated by a low-ID agent must propagate through a longer chain of "bullies" before the highest-ID agent (agent 19) finally wins. This demonstrates a clear scalability cost for this specific election protocol.
- Degraded Formation Accuracy (mean_pos_error : 5.104):** The position error increased compared to the 10-agent test. This is due to two factors:
 - Leader Escape:** The leader is actively moving faster than the followers, making it impossible for them to ever fully close the distance to their target slots.
 - Increased Agent Density:** With 20 agents, the formation is much denser. Followers must constantly make small adjustments to avoid their immediate neighbors, which interferes with their primary goal of reaching their assigned slot. The collision avoidance system is working more frequently, contributing to the position error but to the safety of not colliding with no other agent.

Conclusion from 20-Agent Test: The system remains functional at 20 agents, but the performance shows signs of strain. The leader election takes longer, and formation accuracy suffers due to the combination of the leader's escape maneuver and increased agent density. While the high-level coordination works and they will always follow the leader, the physical realization of the formation becomes more challenging at a larger scale.

Limitations and Future Work

During our experiments, particularly when scaling to a dense formation of 20 agents, we identified a critical limitation in our system: **Leader Entrapment**.

The Leader Entrapment Problem

The collision avoidance system, while essential for safety, is not role-aware. It treats all lidar detections as obstacles to be avoided. In a dense formation, the leader is surrounded by its own followers. When it attempts to move (e.g., to leave the circle or follow a path), its lidar detects the followers as a wall of obstacles. The Time-

To-Collision (TTC) algorithm correctly identifies a high risk of collision and overrides the leader's desired velocity, forcing it to slow down or stop.

Essentially, the leader becomes trapped by the very formation it is trying to lead. The only way we could make the leader "escape" was by making it significantly faster than the followers, which is an inefficient solution that degrades formation accuracy, as seen in our experiments.

Proposed Solution (Future Work)

A more robust solution would be to make the collision avoidance system role-aware. This could be implemented as follows:

1. **Modify the Collision Avoidance Function:** The `_collision_avoidance_vel` function would be updated to accept a boolean flag, `ignore_agents`.
2. **Role-Based Calls:**
 - **The Leader** would call the function with `ignore_agents=True`. In this mode, the function would differentiate between static obstacles (from lidar) and other agents. It would continue to avoid walls but would be programmed to ignore followers, trusting them to get out of its way.
 - **Followers** would call the function with `ignore_agents=False`, ensuring they continue to avoid all other agents and the leader.

This change would give the leader "right of way," allowing it to move freely through the formation while placing the responsibility of collision avoidance on the followers. Implementing this would solve the entrapment problem and allow for much more complex and dynamic group maneuvers.

Code

Available at <https://github.com/Valquaresma03/MAS/tree/main/assignment2>